

Yashirin tartib

IOI saralash IV · 1-kun · A

Alice va Bob permutatsiyalar va qisman tartiblarni o'rganib kelishdi. Ular quyidagi o'yinni o'ylab topishdi.

Yashirin shartlar to'plami mavjud. Har bir shart pozitsiyalar juftligi (a, b) bo'lib, $p_a < p_b$ bo'lishini talab qiladi. Agar a -pozitsiyadagi qiymat b -pozitsiyadagi qiymatdan kichik bo'lsa, p permutatsiya (a, b) shartini qanoatlantiradi. Permutatsiya barcha yashirin shartlarni qanoatlantirsa, u **valid** deyiladi.

O'yin boshida Bob Alicega uzunligi n bo'lgan va valid ekanligi kafolatlangan p permutatsiyani beradi. Alice yashirin shartlarni bilmaydi, lekin boshqa permutatsiyalar haqida savollar berishi mumkin.

Istalgan q permutatsiya uchun Alice q valid yoki yo'qligini tekshirishi mumkin. Natija q barcha yashirin shartlarni qanoatlantirsa `true`, aks holda `false` bo'ladi.

Ushbu ma'lumotdan foydalanib, Alice quyidagilarni aniqlamoqchi:

- leksikografik jihatdan eng kichik valid permutatsiya;
- leksikografik jihatdan eng katta valid permutatsiya.

Sizning vazifangiz Alice rolini bajarishdir.

Implementatsiya tafsilotlari

Quyidagi protsedurani yozishingiz kerak.

```
pair<vector<int>, vector<int>> hidden_order(int n, vector<int> possible_perm)
```

- n : permutatsiya uzunligi.
- `possible_perm`: uzunligi n bo'lgan vektor, 1 dan n gacha butun sonlarning valid permutatsiyasi ($possible_perm[0], \dots, possible_perm[n - 1]$).
- Bu protsedura aynan bir marta chaqiriladi.
- U `{lex_min, lex_max}` vektorlar juftligini qaytarishi kerak; bu yerda `lex_min` leksikografik jihatdan eng kichik valid permutatsiya, `lex_max` esa leksikografik jihatdan eng katta valid permutatsiya.

Grader siz chaqirishingiz mumkin bo'lgan quyidagi protsedurani beradi:

```
bool check_requirements(vector<int> perm)
```

- `perm` : 1 dan n gacha butun sonlarning permutatsiyasi.
- Bu protsedura `perm` barcha yashirin shartlarni qanoatlantirsa `true` , aks holda `false` qaytaradi.
- Uni ko'pi bilan 5 000 marta chaqirish mumkin.

Cheklovlar

- $1 \leq n \leq 100$
- `possible_perm` 1 dan n gacha butun sonlarning valid permutatsiyasi.
- `check_requirements` chaqiruvlari soni 5 000 dan oshmasligi kerak..

Subtasklar

Subtask	Ball	Cheklovlar
1	9	$1 \leq n \leq 6$
2	18	$n \leq 70$, yashirin shartlar soni 1
3	22	$1 \leq n \leq 30$
5	51	Qo'shimcha cheklovlar yo'q.

Misol

Quyidagi chaqiruvni ko'rib chiqing:

```
hidden_order(4, {3, 2, 1, 4})
```

Yashirin shartlar $p_2 < p_1$ va $p_3 < p_4$ bo'lsin.

$(3, 2, 1, 4)$ permutatsiya ikkala shartni ham qanoatlantiradi, shuning uchun u `possible_perm` sifatida berilishi mumkin.

Leksikografik jihatdan eng kichik valid permutatsiya $(2, 1, 3, 4)$, chunki:

- $p_2 = 1 < 2 = p_1$
- $p_3 = 3 < 4 = p_4$

Leksikografik jihatdan eng katta valid permutatsiya $(4, 3, 1, 2)$, chunki:

- $p_2 = 3 < 4 = p_1$
- $p_3 = 1 < 2 = p_4$

Shuning uchun protsedura `{ {2, 1, 3, 4}, {4, 3, 1, 2} }` qaytarishi kerak.

Sample grader

Sample grader inputni quyidagi formatda o'qiydi:

- line 1: n
- line 2: $p_1 p_2 \dots p_n$
- line 3: m — yashirin shartlar soni
- line $4 + i$ ($0 \leq i < m$ uchun): $x_i y_i$ — $p_{x_i} < p_{y_i}$ yashirin shartini bildiruvchi indekslar

Sample grader `hidden_order` ni bir marta chaqiradi va qaytarilgan permutatsiyalarni quyidagi formatda chiqaradi:

- line 1: leksikografik jihatdan eng kichik valid permutatsiyaning probel bilan ajratilgan elementlari
- line 2: leksikografik jihatdan eng katta valid permutatsiyaning probel bilan ajratilgan elementlari

Namunalar

1-namuna

Kirish:

```
4
3 2 1 4
2
2 1
3 4
```

Chiqish:

```
2 1 3 4
4 3 1 2
```